

CO2 ASSESSMENT 2

COURSE: Computer Vision with OpenCV (DSA0201)

CO2: Image Processing, Motion Estimation & Optical Flow (BL3)

NAME: N Yugesh

Reg No:192424329

Question 1: Crack Detection on Concrete Surfaces Using Drone Images

Scenario: A civil inspection team is developing a computer vision system to detect thin cracks on concrete bridges and building surfaces from drone images. Due to changing sunlight, shadows, and camera vibration, the raw images contain uneven illumination, low contrast, and Gaussian noise.

A. Preprocessing pipeline for drone crack-detection images

A complete pipeline should address illumination, noise, contrast, and scale, in the following order:

- 1. Illumination correction: Apply homomorphic filtering or a rolling-ball/top-hat background-subtraction technique (estimate the smooth illumination component with a large Gaussian or morphological opening, then divide/subtract it out). This is needed because uneven sunlight and shadows create large, slow intensity gradients across the surface that are unrelated to actual cracks, and would otherwise be confused with real intensity variations or mask faint cracks in shadowed regions.
- 2. Noise reduction: Apply a Gaussian filter or, preferably, a bilateral filter/non-local means filter. This is needed because the stated Gaussian noise (from vibration and sensor effects) would otherwise generate false gradient responses during edge detection; a bilateral/non-local-means filter is preferred over plain Gaussian blur because it smooths noise while better preserving the thin, low-contrast crack edges that must survive to the detection stage.
- 3. Contrast enhancement: Apply Contrast Limited Adaptive Histogram Equalization (CLAHE). This is needed because cracks are typically low-contrast relative to the surrounding concrete texture; CLAHE enhances local contrast in small tiles (rather than globally), boosting the visibility of fine cracks without over-amplifying noise or saturating already-bright regions.
- 4. Image normalization: Rescale intensity values to a standard range (e.g., 0–1 or 0–255) and, if images are captured at varying drone altitudes/distances, resize to a common resolution/scale. This is needed so that downstream edge detection and any learned/threshold-based models behave consistently across images captured under different lighting and distance conditions.

B. Edge detection technique for thin surface cracks

The Canny edge detector is the most appropriate choice. Cracks are thin, low-contrast, and often noisy in appearance, and Canny's pipeline — Gaussian smoothing, gradient computation, non-maximum suppression, and hysteresis (dual) thresholding — is specifically built to produce thin, well-localized, single-pixel-wide edges while using the low threshold to preserve faint-but-connected crack segments and the high threshold to reject noise.

Comparing against the Sobel operator: Sobel only computes a first-order gradient magnitude image and requires a separate, typically single, threshold to decide what counts as an edge. It produces thick, multi-pixel-wide responses around cracks and has no built-in mechanism to preserve weak-but-continuous features (like a crack that partially fades under a shadow) while rejecting isolated noise. Canny's hysteresis thresholding is specifically better suited here: a crack segment that dips below the noise floor in one shadowed region can still be retained if it connects to a strong-edge segment elsewhere, which a single global Sobel threshold cannot replicate. This makes Canny more suitable for fine, thin, and partially faint cracks.

C. Post-processing to connect broken crack segments

Morphological closing (dilation followed by erosion) using a small, elongated structuring element (e.g., a line-shaped or small $3 \times 3/5 \times 5$ kernel oriented along the expected crack direction) is a suitable post-processing step. Dilation first bridges small gaps between nearby broken edge segments, connecting them into continuous crack lines, and the following erosion then thins the connected regions back toward their original width so the crack does not appear artificially thickened. Alternatively/additionally, a graph-based edge-linking approach (connecting edge pixels within a small search radius and similar orientation) or the Hough transform (for detecting and joining collinear line segments) can be used to explicitly link broken crack fragments into continuous crack paths.

Question 2: Human Motion Tracking in a Security Camera Video

Scenario: *A security camera monitors people moving in different directions in a hallway with plain walls, smooth floors, and occasional shadows. The system must estimate motion of each person across video frames for tracking and activity analysis.*

A. Using Lucas-Kanade optical flow to estimate people's motion

The Lucas-Kanade (LK) method estimates the motion (optical flow vector) of a pixel between two consecutive frames by assuming brightness constancy (a point's intensity does not change between frames, only its position) and local motion constancy (all pixels within a small neighborhood/window move with approximately the same velocity). For each tracked point — typically distinctive features on a person such as corners of clothing, limbs, or head/shoulder contours detected via a corner detector like Shi-Tomasi — LK sets up the optical flow constraint equation ($I_x \cdot u + I_y \cdot v + I_t = 0$) for every pixel in a small window around that point, then solves the resulting overdetermined system using least squares to obtain the displacement vector (u, v). Applying this across consecutive frames for tracked feature points on each person yields their motion trajectory, allowing the system to follow each individual's position and direction as they move through the hallway.

B. Aperture effect and errors in uniform regions

The aperture effect arises because, within a small local window (as used by Lucas-Kanade), only the component of motion perpendicular to a local edge or gradient can be reliably measured — the component parallel to a straight edge or within a uniform (textureless) region is invisible to the local window, since intensity does not change along that direction, making the optical flow equation ill-conditioned or unsolvable there. On plain walls, smooth floors, and uniformly colored clothing, there are few or no strong local gradients in multiple directions, so the algorithm cannot reliably estimate the true 2D motion at those pixels — it may report zero flow, ambiguous flow, or motion only along a spurious direction. This causes gaps or false/inconsistent motion estimates over large uniform body/background regions, even while genuine edges (e.g., silhouette boundaries) still yield accurate flow.

C. Effect of sudden speed/direction change and improving tracking

Lucas-Kanade assumes small, locally consistent motion between consecutive frames. If a person suddenly changes speed or direction, the actual displacement can exceed what the local linear/first-order approximation can capture, causing the optical flow vectors to become inaccurate, lag behind the true motion, or fail entirely (the point may move outside the search window, breaking track continuity).

To improve tracking accuracy in such situations, a pyramidal (multi-scale) Lucas-Kanade implementation should be used: motion is first estimated on a coarse, downsampled image (where the same displacement appears smaller and is easier to capture within a small window) and then progressively refined at finer resolutions, allowing the method to handle larger and more sudden displacements. Additionally, combining optical flow with a predictive tracker such as a Kalman filter (to predict the likely next position based on recent velocity/acceleration and correct for abrupt changes) or a more robust global tracker (e.g., a correlation-based tracker or a deep-learning-based tracker) can further stabilize tracking through sudden motion changes.

Question 3: Vehicle Tracking at a Traffic Junction Using Video Frames

Scenario: A smart traffic monitoring system uses CCTV footage to track vehicles at a busy road junction. Vehicles move at different speeds, some stop suddenly at signals, and others turn left or right. The video also contains shadows, reflections, and partial occlusion when vehicles overlap.

A. Using optical flow to estimate vehicle direction and speed

Optical flow computes a per-pixel (or per-feature-point) displacement vector field between consecutive video frames by tracking how intensity patterns on each vehicle move from one frame to the next. For each vehicle, distinctive feature points (e.g., corners on the vehicle body, edges of windows, headlights) are tracked frame-to-frame; the resulting flow vectors indicate the direction of travel (the vector's orientation) and, when combined with the known frame rate and a calibrated pixel-to-real-world-distance mapping, the speed of the vehicle (vector magnitude $\div$ time between frames, converted to real-world units). Aggregating flow vectors across all points belonging to a single vehicle (e.g., via clustering) gives a robust estimate of that vehicle's overall motion, useful for detecting turning vehicles (direction change) and sudden stops (magnitude drops toward zero).

B. Two challenges affecting motion estimation, and their effects

- Occlusion: When vehicles overlap (e.g., one vehicle passes behind another or they queue closely at a signal), the feature points on the occluded vehicle temporarily disappear from view. This breaks the frame-to-frame correspondence optical flow relies on, causing tracked points to be lost, motion vectors to be assigned incorrectly to the wrong vehicle once it reappears, or tracks to be prematurely terminated and re-initialized as a "new" vehicle, disrupting continuous trajectory estimation.
- Shadows and reflections: Moving shadows (cast by vehicles or surrounding structures) and reflections (e.g., on a wet road or vehicle glass/paint) create apparent intensity changes and edges that do not correspond to actual physical vehicle motion. Optical flow, which relies purely on intensity-pattern changes, can misinterpret a moving shadow as a separate moving object or corrupt the flow vectors of the real vehicle it is attached to, leading to inflated, direction-skewed, or spurious motion estimates.

(Other valid challenges include sudden braking, which can violate the small-motion assumption similarly to Q2C, and camera vibration, which introduces global motion noise across the whole frame.)

C. Improvement method for overlap / sudden direction change

A robust improvement is to combine optical flow with a Kalman filter (or a more general Bayesian tracker) per vehicle: the filter maintains a state estimate of each vehicle's position, velocity, and acceleration, predicts its expected position in the next frame even when its feature points are briefly occluded or the optical flow signal is unreliable, and then corrects that prediction once genuine flow measurements become available again. This allows the tracker to "coast" through short occlusions (bridging the gap when vehicles overlap) and to smoothly re-associate the correct identity to each vehicle afterward, rather than losing or swapping tracks. For frequent, complex overlaps, this can be further strengthened with a multi-object tracking framework (e.g., Kalman filter + Hungarian algorithm for data association, or an appearance-based re-identification model) so that vehicle identities are correctly maintained even through occlusion and abrupt direction changes.